UNIT V – Virtual Functions, Polymorphism & Exception Handling

Complete Answers: Debugging, Corrected C++ Programs, Pseudo-Code, Dry Runs and Outputs

Exam-ready study material prepared from the supplied Unit V question bank. Line numbers below refer to the relevant statements in the supplied buggy snippets; exact editor line numbering may differ.

Quick Concepts for Revision

- Virtual function: enables runtime (late) binding. A call through a base pointer/reference is dispatched to the final overrider of the object's dynamic type.
- vtable/vptr: common implementation technique. A class with virtual functions commonly has a virtual table; each polymorphic object stores a hidden vptr that helps locate the correct function. Exact layout is compiler-dependent.
- Virtual destructor: required when objects may be deleted through a base pointer. It ensures derived destruction occurs before base destruction.
- Pure virtual function: written with = 0; it makes the class abstract unless a derived class implements every inherited pure virtual function.
- Pass by value: copies only the base subobject when a derived object is passed to a base parameter, causing object slicing. Use Base&, const Base&, or Base* for polymorphism.
- Exception flow: throw transfers control to a matching catch handler. Local automatic objects are destroyed during stack unwinding.
- Rethrow: bare throw; preserves the currently handled exception object. throw e; throws a new exception from the static type of e and can slice a derived object.
- Catch ordering: place derived exception handlers before base exception handlers; catch(...) must be last.
- noexcept: a non-throwing contract. If an exception escapes a noexcept function, the program calls std::terminate.

SECTION A – DEBUGGING QUESTIONS

Question 1 – Non-Virtual Destructor in Polymorphic Deletion

Bug classification: Runtime polymorphic-destruction bug; missing virtual destructor. Relevant buggy statement: the Shape destructor declaration (~Shape()) in the supplied class (shown as Line 7).

1. Why the derived destructor does not run

The expression delete shapePtr uses a Shape* whose static type is Shape*. Because Shape::~Shape() is non-virtual, destruction is not dynamically dispatched. The program does not have a valid polymorphic destruction path to ComplexPolygon::~ComplexPolygon().

2. C++ rule

Deleting an object through a base-class pointer when the static base type has a non-virtual destructor is undefined behavior if the object is actually a derived-class object. Therefore, it is not merely a guaranteed base-only deletion; the program has no defined behavior.

3. vtable/vptr explanation

Shape already has a virtual render() function, so it is polymorphic and commonly has a vptr. However, a non-virtual destructor is not selected through virtual dispatch. Making the base destructor virtual gives the implementation a virtual destructor entry, so deleting through Shape* invokes ComplexPolygon::~ComplexPolygon() first and then Shape::~Shape(). The exact vtable layout is implementation-dependent.

4. Corrected C++ code

```
#include <iostream>

class Shape {
public:
    Shape() { std::cout << "Shape created." << std::endl; }

    virtual ~Shape() {
        std::cout << "Shape destroyed." << std::endl;
    }

    virtual void render() const {
        std::cout << "Rendering generic shape." << std::endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double* vertexArray;
    int vertexCount;

public:
    ComplexPolygon(int count) : vertexArray(new double[count * 2]),
                                vertexCount(count) {
        std::cout << "Allocated " << vertexCount
                  << " vertices on heap." << std::endl;
    }

    ~ComplexPolygon() override {
        std::cout << "Deallocating polygon vertex heap memory."
                  << std::endl;
        delete[] vertexArray;
    }

    void render() const override {
        std::cout << "Rendering polygon with " << vertexCount
                  << " vertices." << std::endl;
    }
};

int main() {
    Shape* shapePtr = new ComplexPolygon(1000);
    shapePtr->render();
    delete shapePtr;
    return 0;
}
```

Dry run

Step	Statement / event	Result
1	new ComplexPolygon(1000)	Shape constructor runs; derived object allocates 2,000 doubles.

2	shapePtr->render()	Virtual dispatch selects ComplexPolygon::render().
3	delete shapePtr	Virtual destructor dispatch begins derived destruction.
4	ComplexPolygon destructor	Vertex array is released with delete[].
5	Shape destructor	Base part is destroyed after the derived part.

Expected terminal output

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
Shape destroyed.
```

Note: the original buggy program has undefined behavior at delete shapePtr; therefore, its exact output is not guaranteed.

Question 2 – Inverted Exception Catch Handlers

Bug classification: Compile-time warning/diagnostic for an unreachable or shadowed handler. Relevant statements: base handler first (shown as Line 34), derived handler second (shown as Line 38).

1. Compiler diagnostic

Many compilers warn that the InsufficientFundsException handler is unreachable because the earlier BankingException handler can already catch it. The exact wording depends on the compiler; some compilers issue a warning rather than an error.

2. Catch matching

The thrown object is InsufficientFundsException. It is derived from BankingException, so it can bind to catch (const BankingException&). Since handlers are tested in source order, the base-reference handler matches first and control never reaches the later derived handler.

3. Ordering rule

Place handlers from most specific derived type to most general base type. Place catch(...) last because it matches any exception.

4. Corrected try-catch block

```
int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    catch (const InsufficientFundsException& ife) {
        std::cout << "Shortfall of: $" << ife.getShortfall()
                  << std::endl;
    }
    catch (const BankingException& be) {
        std::cout << "General Banking Error: " << be.getMessage()
                  << std::endl;
    }
    catch (...) {
        std::cout << "Unknown banking failure." << std::endl;
    }
    return 0;
```

```
}
```

Dry run and expected output

Step	Control flow	Result
1	processWithdrawal(500, 750)	Amount exceeds balance by $250.
2	throw InsufficientFundsException	Control leaves processWithdrawal.
3	First catch checked	Derived handler matches exactly.
4	Derived handler executes	Prints shortfall; remaining handlers are skipped.

```
Shortfall of: $250
```

Question 3 – Exception Rethrowing Slicing Bug

Bug classification: Exception object slicing during rethrow. Relevant buggy statement: throw e; (shown as Line 32).

1. Current behavior

The first auditor catch receives the original TimeoutError through a DatabaseError reference and prints the audit message. The statement throw e; creates a new exception using the static type DatabaseError. The derived TimeoutError part, including timeoutMs, is lost. In main(), catch (const TimeoutError&) does not match; catch (const DatabaseError&) handles the new object.

2. Difference between the two forms

- throw e;: throws a new exception object based on the static type of expression e. If e is a base reference to a derived exception, the derived part is sliced.
- throw;: rethrows the currently handled exception object. It preserves its original dynamic type and derived members.

3. Corrected line

```
throw;  // Preserve and rethrow the original TimeoutError object.
```

4. Corrected complete relevant function and expected output

```
void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError& e) {
        std::cout << "[AUDIT LOG]: Intercepted DB error."
                  << std::endl;
        throw; // Correct: rethrows the original exception object.
    }
}
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

For comparison, the buggy version would normally print the audit line followed by the generic DatabaseError client message, because throw e; slices the derived exception.

Question 4 – Instantiating an Incomplete Abstract Class

Bug classification: Compile-time error; VideoPacket remains abstract because it does not override every pure virtual function. Relevant statement: VideoPacket packet(1042) (shown as Line 25).

1. Compilation error

A typical diagnostic is: cannot declare variable 'packet' to be of abstract type 'VideoPacket'. The compiler will identify NetworkPacket::getPacketSize() as a pure virtual function that remains unimplemented. Exact wording varies by compiler.

2. Abstract class condition

A class is abstract if it has at least one pure virtual function for which the class has no final overrider. An abstract class cannot be instantiated directly.

3. When the derived class becomes concrete

A derived class becomes instantiable only after it provides overrides for all inherited pure virtual functions that remain unimplemented.

4. Corrected C++ code

```
#include <iostream>

class NetworkPacket {
public:
    virtual ~NetworkPacket() = default;
    virtual void serialize() = 0;
    virtual int getPacketSize() const = 0;
};

class VideoPacket : public NetworkPacket {
private:
    int frameID;

public:
    VideoPacket(int id) : frameID(id) {}

    void serialize() override {
        std::cout << "Serializing H.264 Video Frame: "
                  << frameID << std::endl;
    }

    int getPacketSize() const override {
        return 1500; // Example packet size in bytes.
    }
};

int main() {
    VideoPacket packet(1042);
    packet.serialize();
    std::cout << "Packet size: " << packet.getPacketSize()
              << " bytes" << std::endl;
    return 0;
}
```

Expected output

```
Serializing H.264 Video Frame: 1042
Packet size: 1500 bytes
```

The value 1500 is an illustrative implementation value because the question does not specify an actual encoded packet size.

Question 5 – Pass-by-Value Object Slicing

Bug classification: Object slicing caused by pass-by-value; runtime polymorphism is lost for the copied parameter. Relevant statement: void triggerEncounter(Entity e) (shown as Line 17).

1. Why the generic function executes

Passing boss to a parameter of type Entity by value constructs a separate Entity parameter from the Entity base subobject of Dragon. The Dragon-specific part is sliced away. The parameter's dynamic type is Entity, so e.attack() calls Entity::attack().

2. vptr explanation

A common implementation stores a hidden vptr in each polymorphic object. The copied parameter is an Entity object, not a Dragon object; its virtual dispatch state corresponds to Entity. A reference parameter does not create a sliced base object and continues to refer to the original Dragon object, allowing the Dragon final overrider to run.

3. Corrected function

```
void triggerEncounter(const Entity& e) {
    e.attack();
}
```

4. Expected output with the corrected function

```
Dragon breathes scorching fire!
```

With the original pass-by-value function, the output is: Entity performs generic scratch.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Question 1 – Universal Payment Gateway with Runtime Polymorphism

Aim: Model multiple payment providers through one abstract interface and dispatch operations at runtime using a base pointer.

Structured pseudo-code

```
ABSTRACT CLASS PaymentGateway
    PURE VIRTUAL METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
    PURE VIRTUAL METHOD Capture() RETURNS BOOLEAN
    VIRTUAL DESTRUCTOR()
END CLASS

CLASS CreditCardGateway INHERITS PaymentGateway
    PRIVATE cardNumber, cvv
    OVERRIDE METHOD Authorize(amount)
        IF cardNumber contains exactly 16 digits AND
           cvv contains exactly 3 digits THEN
            DISPLAY "Card authorization successful"
            RETURN TRUE
        ELSE
            DISPLAY "Invalid card details"
            RETURN FALSE
        END IF
    END METHOD

    OVERRIDE METHOD Capture()
        DISPLAY "Credit-card amount captured"
        RETURN TRUE
    END METHOD
END CLASS

CLASS CryptoGateway INHERITS PaymentGateway
    PRIVATE walletHash
```

```
    OVERRIDE METHOD Authorize(amount)
        IF walletHash begins with "0x" THEN
            DISPLAY "Wallet hash verified"
            IF gas fee is within permitted limit THEN
                DISPLAY "Crypto authorization successful"
                RETURN TRUE
            END IF
        END IF
        DISPLAY "Crypto authorization failed"
        RETURN FALSE
    END METHOD

    OVERRIDE METHOD Capture()
        DISPLAY "Cryptocurrency transfer captured"
        RETURN TRUE
    END METHOD
END CLASS

MAIN
    DECLARE gatewayPtr AS PaymentGateway*
    gatewayPtr = NEW CryptoGateway()

    IF gatewayPtr->Authorize(1500.00) = TRUE THEN
        IF gatewayPtr->Capture() = TRUE THEN
            DISPLAY "Payment completed"
        END IF
    ELSE
        DISPLAY "Payment rejected"
    END IF

    DEALLOCATE gatewayPtr
    // Virtual destructor invokes CryptoGateway then PaymentGateway.
END MAIN
```

Dry-run trace: $1,500 cryptocurrency payment

Step	Call / event	Runtime behavior
1	gatewayPtr = NEW CryptoGateway()	Base pointer stores address of CryptoGateway object.
2	gatewayPtr->Authorize(1500.00)	Late binding dispatches to CryptoGateway::Authorize().
3	Wallet validation	Wallet hash prefix '0x' is checked.
4	Gas validation	Gas fee is checked against the permitted limit.
5	Authorize returns TRUE	Control returns to MAIN.
6	gatewayPtr->Capture()	Dispatches to CryptoGateway::Capture().
7	DEALLOCATE gatewayPtr	Derived destructor runs, then base virtual destructor.

Assumption: the sample trace assumes a valid wallet hash and an acceptable gas fee. The scenario does not provide a real wallet or gas-fee value.

Question 2 – Resilient Banking Engine with Nested Try-Catch and Rethrowing

Structured pseudo-code

```
CLASS TransactionException
```

```
    ATTRIBUTE reason AS STRING
    CONSTRUCTOR(reason)
        SET this.reason = reason
    END CONSTRUCTOR
END CLASS

CLASS AccountFrozenException INHERITS TransactionException
    CONSTRUCTOR(reason)
        CALL BASE CONSTRUCTOR(reason)
    END CONSTRUCTOR
END CLASS

CLASS DailyLimitExceededException INHERITS TransactionException
    ATTRIBUTE excessAmount AS REAL
    CONSTRUCTOR(reason, excess)
        CALL BASE CONSTRUCTOR(reason)
        SET excessAmount = excess
    END CONSTRUCTOR
END CLASS

FUNCTION DebitLedger(amount, isFrozen, dailyTotal)
    IF isFrozen = TRUE THEN
        THROW NEW AccountFrozenException("Account is frozen")
    END IF

    IF dailyTotal + amount > 50000 THEN
        SET excess = dailyTotal + amount - 50000
        THROW NEW DailyLimitExceededException(
            "Daily withdrawal limit exceeded", excess)
    END IF

    DISPLAY "Ledger debited"
END FUNCTION

FUNCTION AuditTransactionWrapper(amount, isFrozen, dailyTotal)
    TRY
        CALL DebitLedger(amount, isFrozen, dailyTotal)
    CATCH (e AS TransactionException)
        WRITE "SECURITY AUDIT: " + e.reason
        RETHROW
    END TRY
END FUNCTION

MAIN
    TRY
        TRY
            CALL AuditTransactionWrapper(1000, TRUE, 0)
        CATCH (e AS TransactionException)
            DISPLAY "Intermediate transaction handling"
            RETHROW
        END TRY

    CATCH (e AS AccountFrozenException)
        DISPLAY "Transaction rejected: account is frozen"
    CATCH (e AS DailyLimitExceededException)
        DISPLAY "Daily limit exceeded"
        DISPLAY e.excessAmount
    CATCH (e AS TransactionException)
        DISPLAY "General transaction error: " + e.reason
```

```
    CATCH (...)
        DISPLAY "Unknown system error"
    END TRY
END MAIN
```

Dry-run trace: withdrawal of $1,000 on a frozen account

Step	Control location	Event / result
1	MAIN	Calls AuditTransactionWrapper(1000, TRUE, 0).
2	AuditTransactionWrapper	Enters its TRY block and calls DebitLedger.
3	DebitLedger	Detects isFrozen = TRUE.
4	DebitLedger → throw	Throws AccountFrozenException; normal ledger processing stops.
5	Stack unwinding	Local automatic objects in exited scopes are destroyed, if any.
6	Wrapper catch	TransactionException reference catches the derived exception and writes audit log.
7	RETHROW	Bare rethrow preserves AccountFrozenException's dynamic type.
8	MAIN inner catch	Base TransactionException handler catches, displays intermediate message, and rethrows.
9	MAIN outer catch	AccountFrozenException handler matches before the base handler.
10	Program continues	Displays frozen-account rejection; later handlers are skipped.

Illustrative output

```
SECURITY AUDIT: Account is frozen
Intermediate transaction handling
Transaction rejected: account is frozen
```

Question 3 – Polymorphic Game Engine with Virtual Destructors

Structured pseudo-code

```
ABSTRACT CLASS GameObject
    PURE VIRTUAL METHOD Update(deltaTime AS REAL)
    PURE VIRTUAL METHOD Render()
    VIRTUAL DESTRUCTOR()
END CLASS

CLASS ParticleSystem INHERITS GameObject
    PRIVATE particleCoordinates AS ARRAY OF REAL*
    PRIVATE particleCount AS INTEGER

    CONSTRUCTOR(count)
        particleCount = count
        particleCoordinates = ALLOCATE ARRAY(count * 2)
    END CONSTRUCTOR

    OVERRIDE METHOD Update(deltaTime)
        UPDATE every particle position using deltaTime
```

```
    END METHOD

    OVERRIDE METHOD Render()
        DISPLAY particle system
    END METHOD

    DESTRUCTOR()
        DEALLOCATE particleCoordinates
        DISPLAY "Particle memory released"
    END DESTRUCTOR
END CLASS

MAIN
    DECLARE sceneEntities AS ARRAY OF GameObject*
    sceneEntities[0] = NEW ParticleSystem(100)
    sceneEntities[1] = NEW ParticleSystem(50)

    FOR EACH entity IN sceneEntities
        entity->Update(0.016)
        entity->Render()
    END FOR

    FOR i = 0 TO LENGTH(sceneEntities) - 1
        DEALLOCATE sceneEntities[i]
    END FOR
END MAIN
```

Cleanup order

1. DEALLOCATE sceneEntities[i] invokes the virtual destructor through GameObject*.
2. ParticleSystem destructor executes first and releases the dynamically allocated particle-coordinate array.
3. After the derived destructor body finishes, the GameObject base destructor executes.
4. The next pointer in the array is deleted in the same way.

Key point: destruction order is derived class first, base class second. Without a virtual base destructor, deleting a derived object through GameObject* would be undefined behavior.

Pointer	Derived cleanup	Base cleanup
sceneEntities[0]	ParticleSystem destructor releases 200 coordinate values.	GameObject destructor.
sceneEntities[1]	ParticleSystem destructor releases 100 coordinate values.	GameObject destructor.

Question 4 – File Stream Pipeline with Stack Unwinding and Catch-All

Structured pseudo-code

```
CLASS ResourceGuard
    PRIVATE handle
    CONSTRUCTOR(fileName)
        handle = OPEN fileName
        IF handle is invalid THEN
            THROW FileOpenException("Cannot open file")
        END IF
        DISPLAY "Resource acquired"
    END CONSTRUCTOR

    DESTRUCTOR()
```

```
        IF handle is valid THEN
            CLOSE handle
            DISPLAY "Resource released"
        END IF
    END DESTRUCTOR
END CLASS

CLASS CRCValidationException
    ATTRIBUTE expectedChecksum, calculatedChecksum
    CONSTRUCTOR(expected, calculated)
        SET expectedChecksum = expected
        SET calculatedChecksum = calculated
    END CONSTRUCTOR
END CLASS

FUNCTION IngestDataStream(fileName)
    DECLARE guard AS ResourceGuard(fileName)  // automatic local object
    READ data blocks
    CALCULATE calculatedCRC
    IF calculatedCRC != expectedCRC THEN
        THROW NEW CRCValidationException(expectedCRC, calculatedCRC)
    END IF
    DISPLAY "File ingestion successful"
END FUNCTION

MAIN
    TRY
        CALL IngestDataStream("mission_data.bin")
    CATCH (e AS CRCValidationException)
        DISPLAY "CRC validation failed"
        DISPLAY expected and calculated checksums
    CATCH (...)
        DISPLAY "Unknown file-ingestion failure"
    END TRY
END MAIN
```

Trace: CRC mismatch

Order	Event	Explanation
1	Enter IngestDataStream()	A local ResourceGuard object is constructed.
2	ResourceGuard constructor	File handle is acquired.
3	Read and calculate CRC	Calculated checksum differs from expected checksum.
4	THROW CRCValidationException	Control exits the function; normal statements after throw are skipped.
5	Stack unwinding	The local ResourceGuard destructor runs automatically.
6	Resource release	The file/hardware handle is closed before the catch handler begins.
7	Main specific catch	CRCValidationException handler matches and reports checksum mismatch.
8	Catch-all	Not executed because the specific handler already matched.

Illustrative output

```
Resource acquired
Resource released
CRC validation failed
Expected checksum: 12345
Calculated checksum: 98765
```

The checksum numbers above are illustrative only; the question does not supply actual checksum values. If opening the file itself fails, a FileOpenException would be handled by the catch-all unless a specific handler is added.

Question 5 – Scientific Telemetry Calibration with noexcept Contracts

Structured pseudo-code

```
CLASS OutOfBoundsException
    ATTRIBUTE measuredTemperature AS REAL
    CONSTRUCTOR(temp)
        measuredTemperature = temp
    END CONSTRUCTOR
END CLASS

FUNCTION CalibrateThermalSensors(rawVoltage) RETURNS REAL
    // Potentially throwing: noexcept(FALSE)
    temperature = CONVERT_VOLTAGE_TO_TEMPERATURE(rawVoltage)

    IF temperature < -273.15 OR temperature > 1500.0 THEN
        THROW NEW OutOfBoundsException(temperature)
    END IF

    RETURN temperature
END FUNCTION

FUNCTION CommitTelemetry(finalTemp) NOEXCEPT
    TRY
        WRITE finalTemp TO avionics hardware bus
    CATCH (...)
        LOG OR SUPPRESS hardware communication failure
        // Do not allow any exception to escape this NOEXCEPT function.
    END TRY
END FUNCTION

MAIN
    TRY
        temperature = CalibrateThermalSensors(faultyVoltage)
        CALL CommitTelemetry(temperature)
        DISPLAY "Telemetry committed"
    CATCH (e AS OutOfBoundsException)
        DISPLAY "Calibration rejected: temperature outside safe range"
        DISPLAY e.measuredTemperature
    CATCH (...)
        DISPLAY "Unexpected telemetry error"
    END TRY
END MAIN
```

Dry-run trace: faulty reading produces -300.0°C

Step	Function / event	Result
1	MAIN calls CalibrateThermalSensors()	Calibration begins.
2	Voltage conversion	Computed temperature is -300.0°C.
3	Boundary validation	-300.0 is below the minimum -273.15°C.

4	THROW OutOfBoundsException	Calibration function exits by throwing.
5	Stack unwinding	Automatic local objects in the exited function are destroyed.
6	MAIN catch	OutOfBoundsException handler displays the invalid temperature.
7	CommitTelemetry()	Not called because calibration did not return a valid temperature.

Illustrative output

```
Calibration rejected: temperature outside safe range
Measured temperature: -300.0 C
```

Important C++ note: a function declared noexcept must not allow an exception to escape. Catching all exceptions inside CommitTelemetry is one way to uphold that contract. If an exception escapes anyway, std::terminate is called. The historical dynamic exception specification syntax such as throw(Type) is deprecated and removed in modern C++; noexcept is the modern specification.

Final Unit V Revision – Common Exam Points

Topic	Remember
Dynamic dispatch	Use virtual function + base pointer/reference; derived override executes.
Virtual destructor	Declare virtual in polymorphic base classes when deleting through base pointers.
Abstract class	At least one pure virtual function without a final overrider; cannot instantiate.
Object slicing	Avoid base-class pass-by-value for polymorphic objects; use reference or pointer.
Catch order	Derived exception handlers before base handlers; catch(...) last.
Rethrow	Use bare throw; inside a catch to preserve the original exception.
Stack unwinding	Automatic local objects are destroyed as exception propagation exits scopes.
RAII	Acquire resources in constructors and release them in destructors.
noexcept	No exception may escape; violation calls std::terminate.

Exam writing tip: For each debugging answer, state (1) bug classification and relevant line, (2) reason, (3) corrected code, (4) dry-run sequence, and (5) output. For pseudo-code answers, show class relationships, virtual/pure virtual methods, exception flow, and a trace table.